

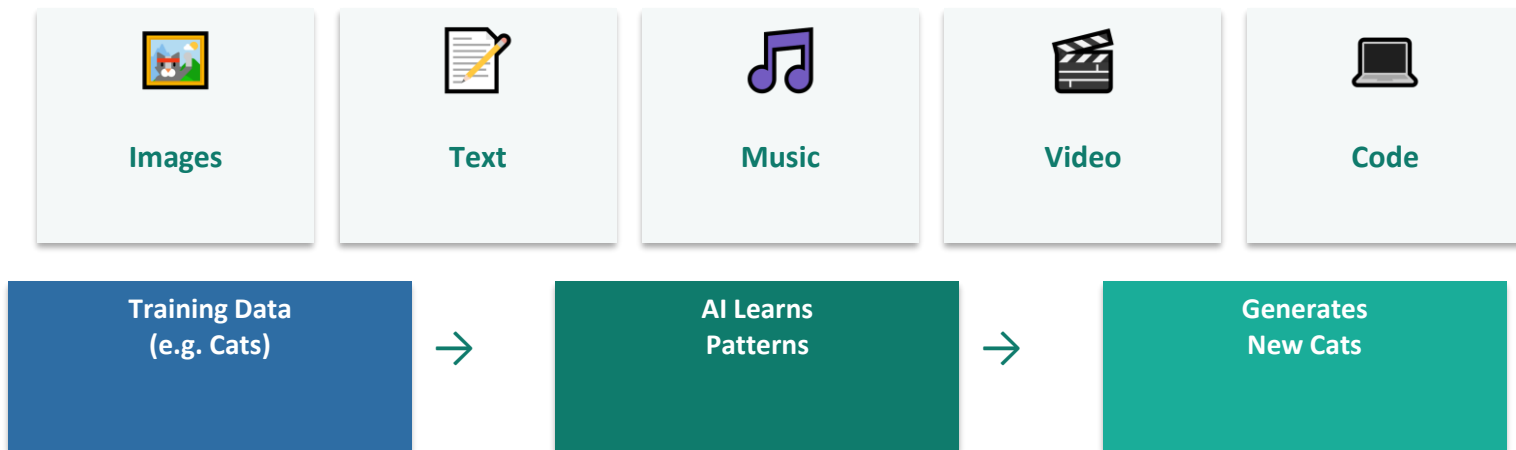
Introduction to

Generative AI

Types · Motivation · Diffusion Models

What is Generative AI?

Generative AI learns patterns from existing data and creates new, realistic content with similar characteristics.



Generative AI does not memorize — it learns patterns to create entirely new content.

Discriminative vs Generative Models

Discriminative Model

recognize

- ▶ Predicts labels or categories
- ▶ Learns conditional probability
- ▶ Classic use: classification
- ▶ Examples: CNN, ResNet, BERT

Image → Cat or Dog?

Generative Model

create

- ▶ Creates new, realistic data
- ▶ Learns joint probability distribution “Learns **how the data and labels are related together.**”
- ▶ Classic use: data generation
- ▶ Examples: GAN, VAE, Diffusion

Noise → Generate a Cat 🐱

Discriminative → classifies existing data. Generative → creates new data from scratch.

Major Types of Generative Models

VAE

Variational Autoencoder

Encode → Latent Space →
Decode

GAN

Generative Adversarial Network

Generator vs. Discriminator

Autoregressive

Autoregressive Models

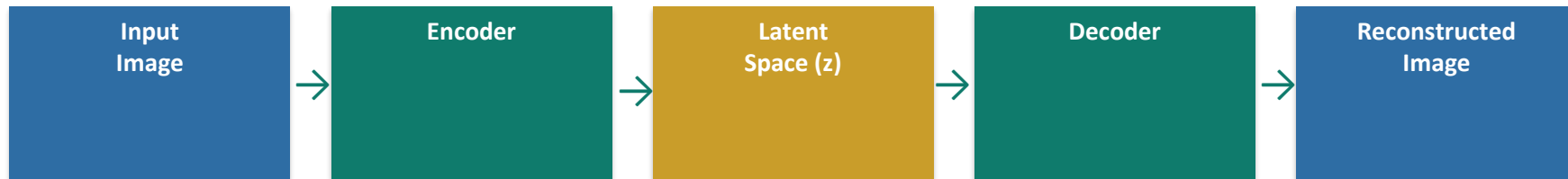
Predict one token/pixel at a
time

Diffusion

Diffusion Models

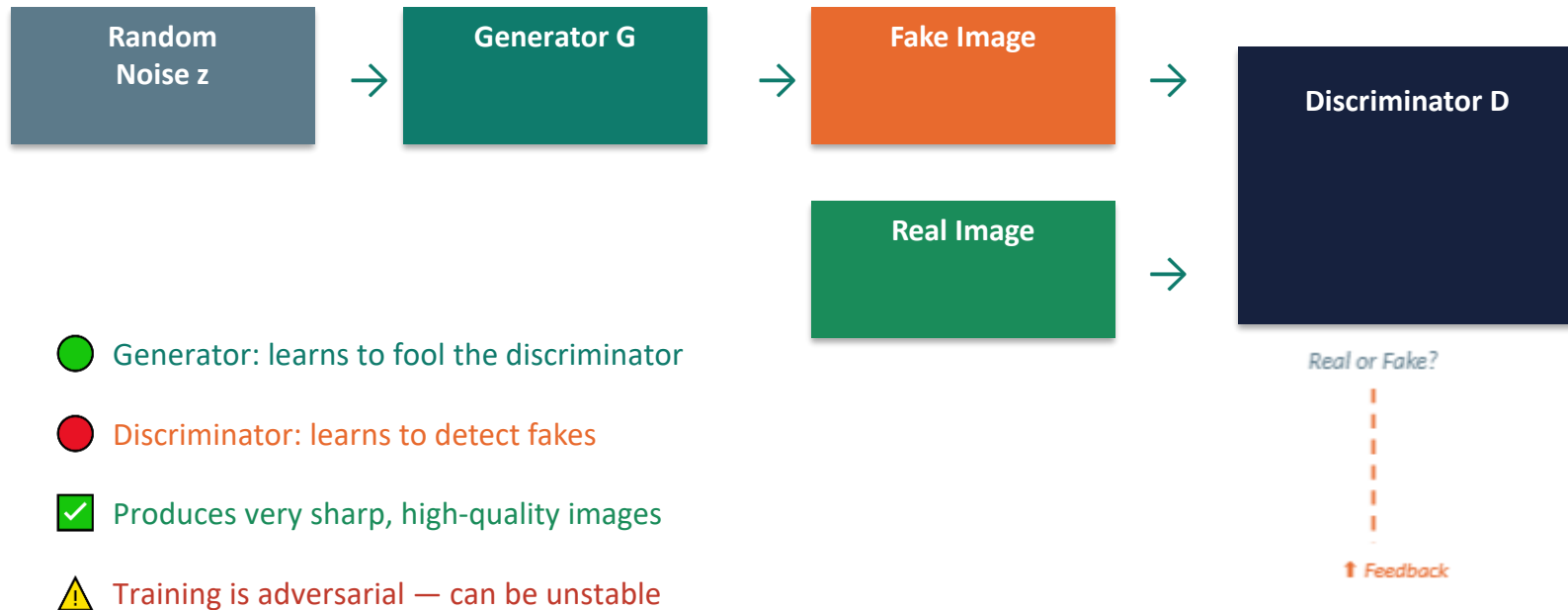
Denoise from pure noise →
image

Variational Autoencoder (VAE)



- Compresses input into a compact latent space representation
- Unlike a normal autoencoder, a VAE doesn't store **one exact point**. Instead, it learns a **distribution** of possible points.
- Samples from the latent space to generate new, similar examples
- ⚠ Produces smooth but sometimes blurry outputs
- ✅ Very stable to train — uses a well-defined reconstruction loss

Generative Adversarial Network (GAN)



GAN: adversarial game produces sharp images, but training is notoriously unstable.

Autoregressive Models



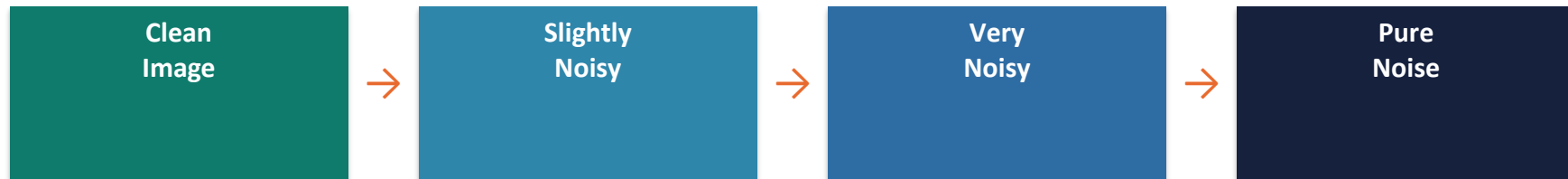
Each step predicts the next token given all previous tokens (context window)

- ▶ The backbone of large language models (GPT, Gemini, etc.)
- ▶ Also applied to images: generate pixels sequentially (PixelCNN)
- ▶ Slow at inference — must generate one token at a time

Autoregressive: predict next token given past context — the engine of modern LLMs.

Diffusion Models

Forward Process (Training: add noise)



Reverse Process (Inference: remove noise step by step)



- ✦ Trained to predict (and reverse) a gradual Gaussian noising process
- ✦ Produces highly realistic, diverse images — current state-of-the-art
- ✦ More stable than GANs — no adversarial competition, simple MSE loss

Diffusion = learn to reverse noise. The basis of DALL·E 3, Stable Diffusion, Imagen.

Comparing Generative Models

Model	Speed	Image Quality	Training Stability
VAE	Fast	Medium	High
GAN	Fast	High	Low
Autoregressive	Slow	High	High
Diffusion	Slow	Excellent	High

Diffusion models are slowest during inference but consistently produce the best visual quality.

Diffusion = best quality + stable training, but slower. GAN = fast but unstable.

Motivation Behind

Diffusion Models

Why GANs weren't enough · The case for stable training

Challenges of GANs

Unstable Training



Generator and Discriminator continuously compete. If one outpaces the other, gradients vanish and training collapses.

Mode Collapse



The generator produces only a narrow set of outputs (e.g. always the same face) instead of capturing full data diversity.

Difficult Optimization



Two losses must stay balanced. If D gets too strong, G gets no useful gradient signal. Tuning is painful.

GANs struggle with instability, mode collapse, and a delicate two-loss balancing act.

Advantages of Diffusion Models



Stable Training

Simple supervised denoising loss (MSE). No adversarial competition — no balancing act required.



Excellent Quality

State-of-the-art visual fidelity. Powers DALL·E 3, Stable Diffusion, and Imagen.



High Diversity

Same prompt → many different valid outputs. No mode collapse by design.



Scales Easily

Performance improves consistently with more data, bigger models, and more compute.



Flexible Conditioning

Text, image, or class labels can steer generation via classifier-free guidance.

Diffusion wins on every front: quality, diversity, stability, and scalability.

GAN vs Diffusion — Final Comparison

Feature	GAN	Diffusion
Training	Difficult	Stable
Image Quality	High	Excellent
Diversity	Medium	High
Mode Collapse	Yes	No
Training Speed	Fast	Slower
Sampling Speed	Fast	Slower
Scalability	Moderate	Excellent

Diffusion models became dominant because they combine stable training, exceptional quality, and strong diversity — making them the foundation of modern AI image generation.

TASK

Build a **Generative Adversarial Network (GAN)** that learns the Fashion-MNIST dataset and generates new clothing images from random noise.

Diffusion Models

Forward Diffusion (Noise Addition) & *Reverse Diffusion* (Denoising)



Topic 3

Forward Process



Topic 4

Reverse Process



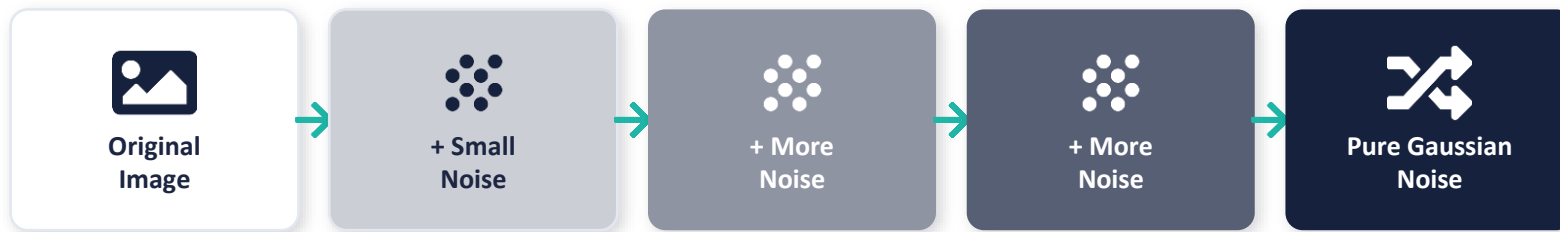
Core Idea

Learn to Denoise

A structured walkthrough of how diffusion models add and remove noise to generate images

Forward Diffusion Process

Gradually corrupting a clean image with Gaussian noise until nothing remains



Fixed process — not learned



Adds noise over many timesteps



Creates the training data for the model

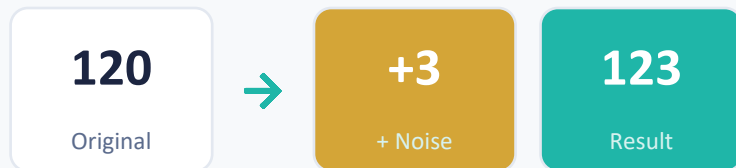


Eventually destroys all image information

What is Gaussian Noise?

Random noise sampled from a normal distribution, applied pixel by pixel

A SINGLE PIXEL



$$\epsilon \sim N(0, I)$$

Noise sampled from a standard normal distribution

PROPERTIES

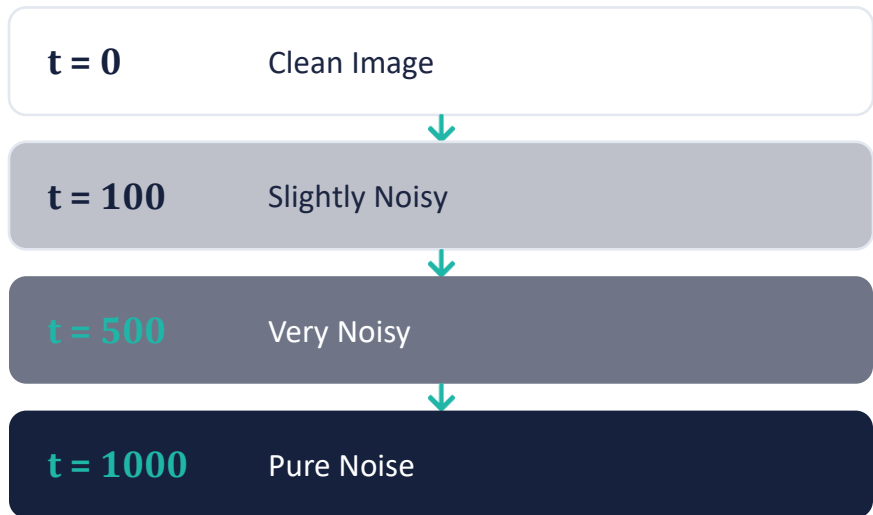
- Mean = 0 , the noise doesn't make the image brighter or darker.
- I = Identity covariance matrix, meaning each pixel's noise is sampled **independently** with the same variance.
- Random positive and negative values
- Smooth randomness
- Easy to generate mathematically

WHY GAUSSIAN?

Natural statistical properties, easy to analyze mathematically, stable optimization, and independent random samples.

Timesteps & Noise Schedule

The forward process unfolds across many discrete steps, $T = 1000$ commonly



β controls how much noise is added at each step — no need to derive the formulas yet.

Forward Diffusion as a Markov Chain

Each state depends only on the one immediately before it



$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

The noisy image at step t depends only on the image at step $t-1$, not on any earlier step.

Markov Property — the future depends only on the present, not the past.

Forward Diffusion Equation

Every symbol explained — no derivation needed

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = N(\sqrt{(1-\beta_t)} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

 $\mathbf{x}_{t-1}$

Previous image

 $\mathbf{x}_t$

Current noisy image

 β

Noise amount

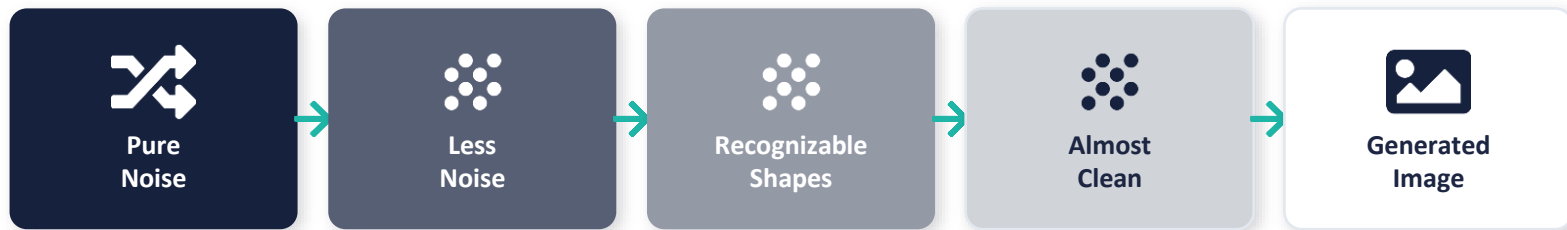
 N

Gaussian distribution

Every timestep: keep most of the image, add a little Gaussian noise.

Reverse Diffusion Process (Denoising)

Instead of adding noise, we remove it step by step — and this is learned



Forward = Fixed

Known equations, no training needed



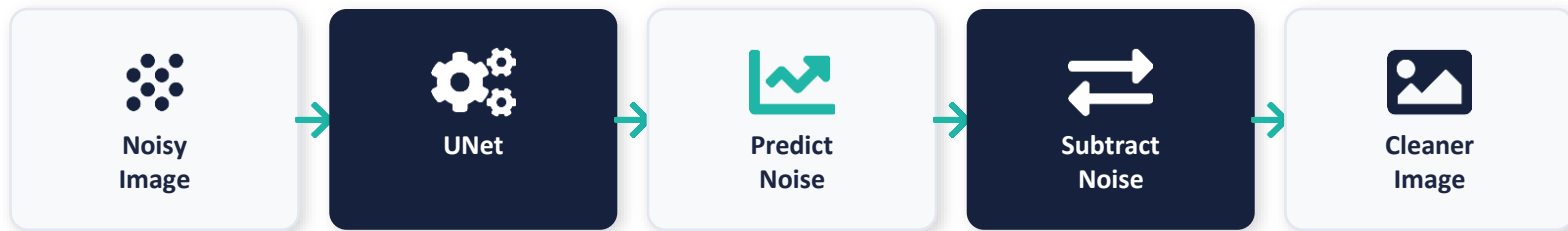
Reverse = Learned

A neural network learns to undo each step

Reverse diffusion turns pure noise back into a realistic image, one small step at a time.

How Does the Model Learn?

The network predicts the noise that was added, then subtracts it



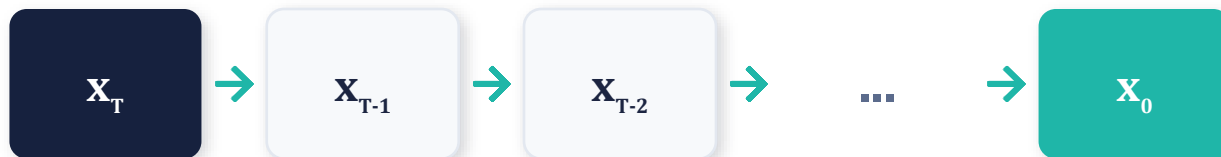
The model does not directly generate images. Instead, it learns: **“What noise was added?”** Then it subtracts that noise.

Predicting noise is a far easier learning target than predicting the clean image outright.

Predict the noise, subtract it — this single reframing is what makes diffusion models trainable.

Reverse Markov Chain

Unlike the forward process, the reverse probabilities are unknown — and must be learned



$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$



Forward process: **Known**



Reverse process: **Learned**

The reverse Markov chain mirrors the forward one — but its transitions must be approximated.

Reverse Diffusion Equation

The same expression as before — now broken down symbol by symbol

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

p

Learned probability

θ

Neural network parameters

x_t

Current noisy image

x_{t-1}

Cleaner image

Forward process is known by formula. Reverse process is learned by the network.

Forward vs. Reverse Diffusion

A side-by-side summary of the two processes covered today



FORWARD DIFFUSION

Add noise

Fixed process

Uses Gaussian noise

Destroys information



REVERSE DIFFUSION

Remove noise

Learned process

Uses a neural network

Reconstructs information

Forward: $q(x_t \mid x_{t-1})$

SUMMARY

The Complete Pipeline

Training and inference, side by side

TRAINING

- 1 Image
- 2 Forward Diffusion (Add Noise)
- 3 Random Noise
- 4 Train UNet to Predict Noise

INFERENCE

- 1 Random Noise
- 2 UNet
- 3 Remove Noise
- 4 Generated Image

Train by adding noise and predicting it. Generate by starting from noise and removing it.

DIFFUSION MODELS

Training Objective & Sampling Algorithms

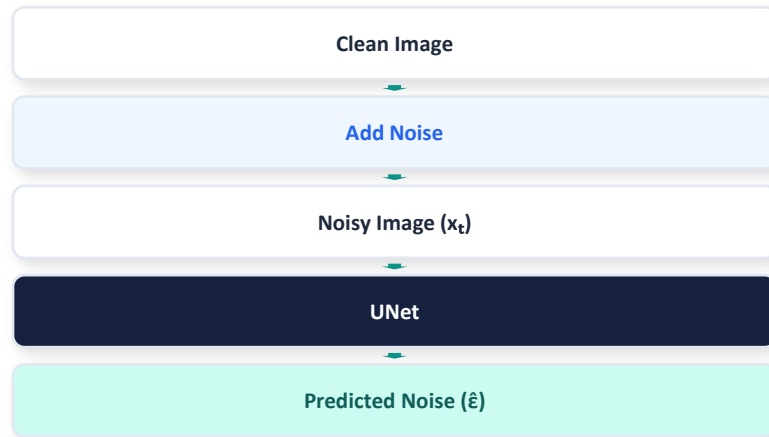
From predicting noise to generating a finished image, step by step.

What Does the Model Learn?

During training we already know the original image, the noise we added, and the resulting noisy image — so the network just has to predict that known noise.

✗ Common assumption: predict the final image

✓ Actually true: predict the noise that was added



compared against the true noise ϵ during training

Known noise + known image → the network simply learns to reverse the noising step.

Why Predict Noise Instead of the Image?

Reconstructing millions of exact pixel values is a much harder learning problem than predicting a value drawn from a known distribution.

Predict Image

- ✗ Difficult
- ✗ Large search space
- ✗ Less stable
- ✗ Lower performance

Predict Noise

- ✓ Easier
- ✓ Follows a known Gaussian distribution
- ✓ More stable
- ✓ Better image quality

The noise follows a simple Gaussian — far easier to learn than reconstructing an image pixel-by-pixel.

Three Prediction Targets

Modern diffusion models don't always predict the same quantity.



ϵ

Noise Prediction

Estimate the noise added to the image. Most common approach.

DDPM, Stable Diffusion



x_0

Image Prediction

Instead of noise, directly estimate the original clean image.

Some diffusion variants



v

Velocity Prediction

A combination of image and noise — faster convergence, more stable training, better sampling.

Stable Diffusion XL and newer models

Training Loss

The objective is simple: make the predicted noise as close as possible to the actual noise.

$$L = \|\epsilon - \hat{\epsilon}\|^2$$

Mean Squared Error (MSE) between true and predicted noise

Symbol	Meaning
ϵ	True noise
$\hat{\epsilon}$	Predicted noise
L	Error

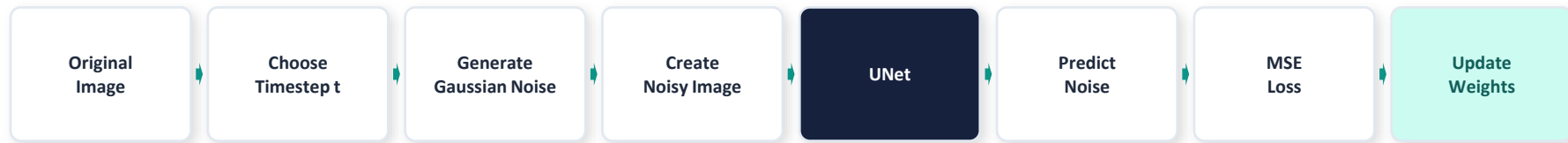
Why the error is small

Position	True Noise	Predicted	Diff
1	0.10	0.12	0.02
2	0.20	0.18	0.02
3	0.40	0.35	0.05

Small error → Good model

Complete Training Pipeline

Every step in one pass, from a clean image to an updated network.

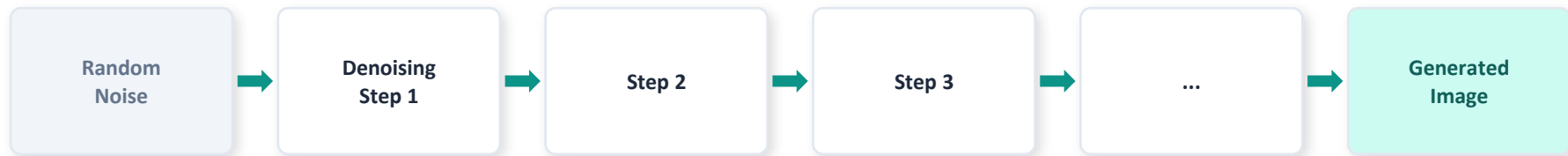


Repeats for many images and many random timesteps until the model converges.

One full pass: noise an image, ask the UNet to predict that noise, then correct the weights.

Image Generation (Sampling)

Training is finished. Sampling means running the reverse diffusion process — no learning happens here, the trained model is used only for inference.



Sampling = the reverse process. The UNet only predicts noise; it never sees the final image directly.

Why Different Samplers?

If every sampler removes noise, why do we need several? Because each makes a different trade-off between speed and quality.

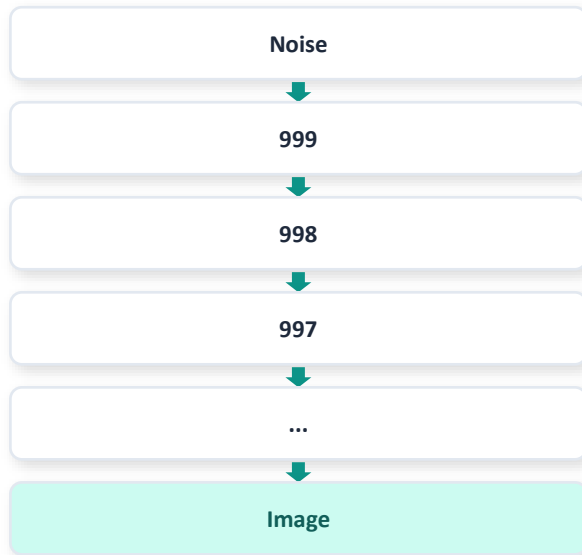
Sampler	Speed	Quality
DDPM	Slow	Excellent
DDIM	Faster	Very Good
Euler	Fast	Good
Heun	Medium	Better than Euler
DPM Solver	Very Fast	Excellent

All samplers solve the same problem — moving from pure noise to a clean image — using different numerical strategies.

DDPM — Denoising Diffusion Probabilistic Model

- Original diffusion algorithm
- Removes a tiny amount of noise every step
- Usually 1000 denoising steps
- Highest quality
- Slow inference

1000 steps, one tiny denoise at a time



DDPM is the gold standard for quality — but its 1000-step process is the slowest way to sample.

DDIM — Skipping Steps

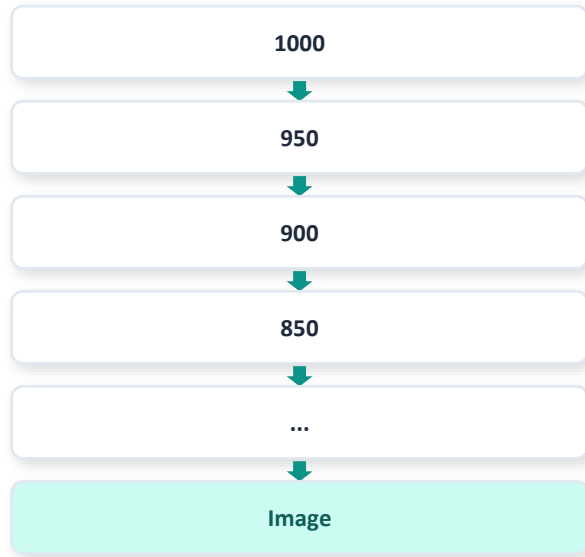
Idea: skip some denoising steps. Instead of 1000 steps, use maybe 50 or even 25.

✓ Faster

✓ Nearly same quality

✓ Deterministic sampling

From 1000 steps down to a handful



DDIM trades a handful of steps for nearly the same quality — and runs deterministically.

Modern Samplers

These methods treat diffusion as solving a differential equation — designed for high quality with fewer denoising steps.

Euler

Simple numerical solver — the most basic ODE step.

Heun

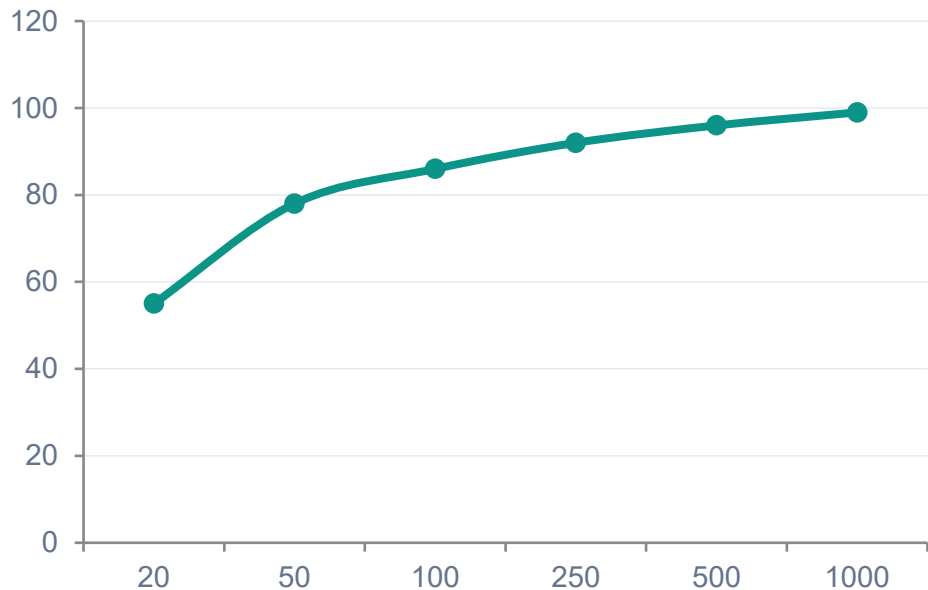
Corrects Euler's estimate for better accuracy.

DPM Solver

Higher-order ODE solver requiring fewer steps.

Inference Steps: Quality vs. Speed

One of the most important practical knobs in diffusion sampling.



20 steps

Very fast, lower quality

50 steps

Good balance

100 steps

Better quality

1000 steps

Excellent, very slow

Quality vs. Speed

More Steps

- Higher quality
- More computation
- Longer runtime

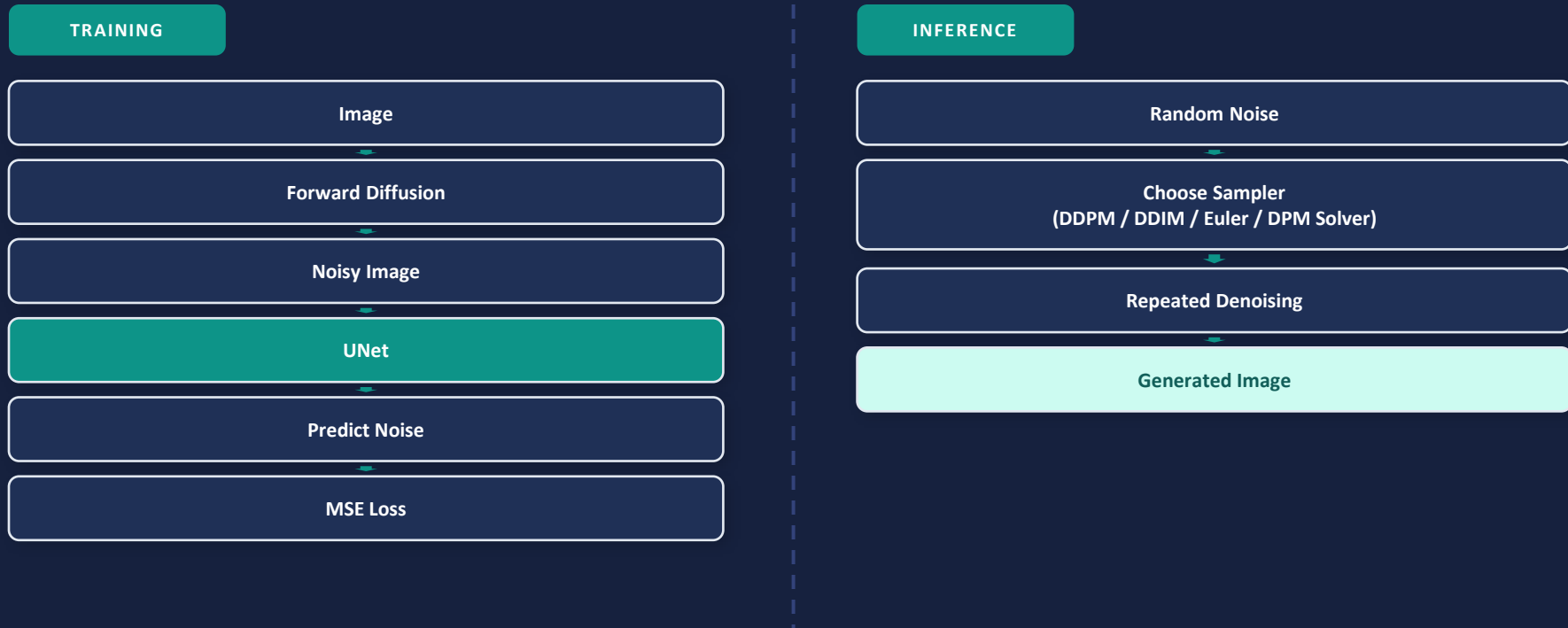
Fewer Steps

- Faster generation
- Less computation
- Shorter runtime

There is no free lunch — choose the step count that fits your speed and quality budget.

Putting It Together

Training teaches the model to predict noise. Sampling reverses that process to generate a brand-new image.



Latent Diffusion Models, Text Conditioning & CFG

Latent Spaces · Prompt Embeddings · Cross-Attention · Classifier-Free Guidance · Applications

Why Latent Diffusion?

THE PROBLEM

$1024 \times 1024 \times 3$ pixels

= 3,145,728 values

-  Slow training & inference
-  Massive memory footprint
-  Computationally expensive
-  Hard to scale to high res

THE SOLUTION

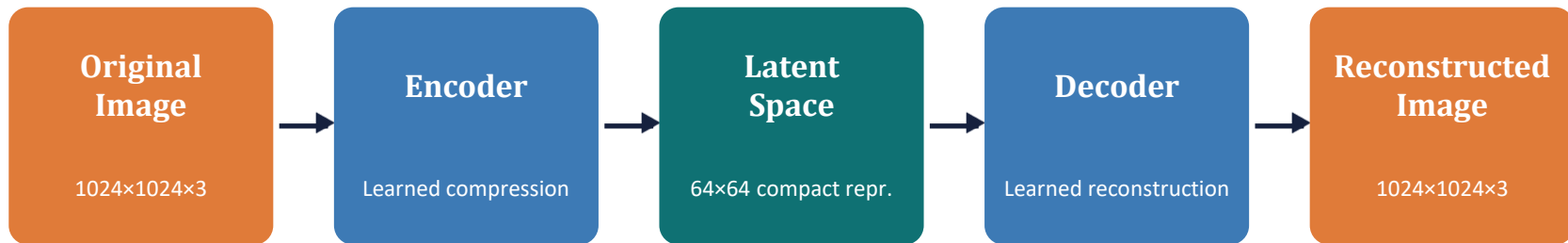
Work in Latent Space

64 × 64 latent

-  Up to 64× fewer values
-  Dramatically less memory
-  Much faster to train
-  Supports high resolutions

Key Idea: Compress first, then diffuse — work in a compact latent space instead of raw pixels.

What is Latent Space?



Compression

The encoder reduces millions of pixel values to thousands of latent values while retaining visual meaning.

Key Insight

Similar images produce similar latent representations — the space is semantically structured.

Efficiency

Stable Diffusion uses a 4×64×64 latent instead of 3×512×512 pixels — 48× fewer values.

Latent space = compressed representation that keeps what matters, discards what doesn't.

The Latent Diffusion Pipeline



PIXEL SPACE

LATENT SPACE (where diffusion happens)

PIXEL SPACE



Speed

Diffusion on 64×64 latents is orders of magnitude faster than on 1024×1024 pixels



Memory

Training fits in consumer-grade GPUs — democratizes generative AI research



Quality

Decoding from rich latents produces sharp, high-fidelity images at any resolution



Flexible

The decoder can upsample to 2K, 4K output without retraining the core model

Pixel Diffusion vs Latent Diffusion

	Pixel Diffusion	Latent Diffusion
 Speed	Slow — millions of values per step	Fast — compact latent vectors
 Memory	Huge GPU memory required	Fits in consumer hardware
 Cost	Expensive to train & run	Efficient, accessible
 Resolution	Hard to scale beyond 512px	Scales to 4K+ via decoder
 Examples	Early DDPM, ADM	Stable Diffusion, Flux

Latent diffusion is the architecture behind Stable Diffusion — the most widely used open-source image model.

Text Conditioning

How does a prompt control image generation?

 "A cute cat wearing sunglasses on a beach at sunset"



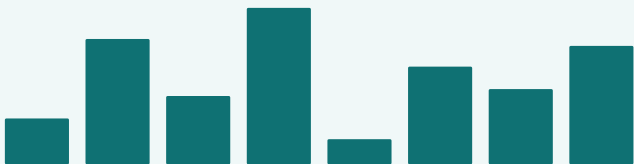
Text cannot enter the model directly — it must first become numerical embeddings that the UNet can process.

Prompt Embeddings

What is an Embedding?

An embedding is a numerical vector that encodes the meaning of text. Similar concepts produce vectors that are close together in the embedding space.

"A cute cat" →



[0.25, 0.71, 0.38, 0.89 ...]

Semantic Similarity

Similar prompts → nearby vectors

"cat" — reference

"kitten" — very close

"small cat" — close

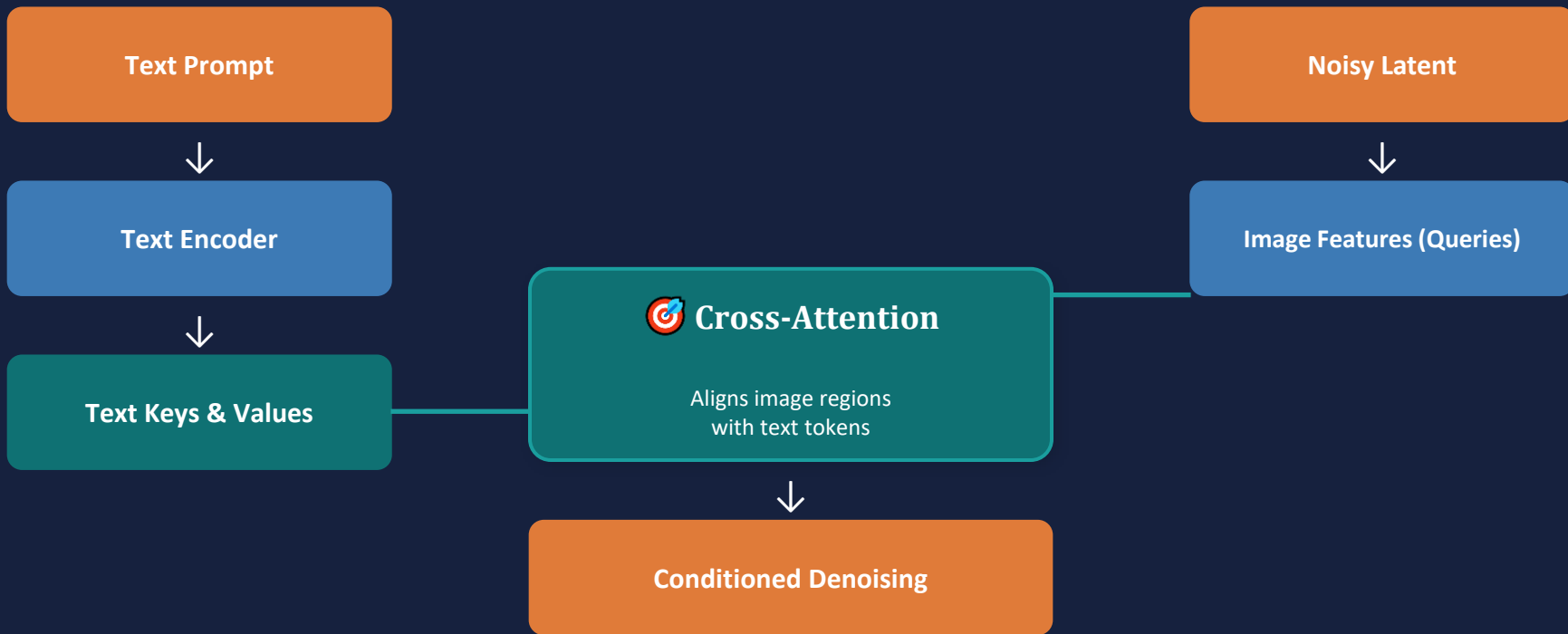
"feline" — nearby

"dog" — far away

Embeddings capture meaning — they allow the model to understand relationships between concepts.

Cross-Attention

The bridge between language and vision



Example: "red sports car" → "red" guides color · "sports car" guides shape & structure

Conditioning — Guiding Generation

Conditioning means providing extra information to steer the output.



Text Prompt

Describe the image in natural language.
"A forest in autumn at golden hour"



Reference Image

Provide an image as style or structure guidance.
Img2Img workflows.



Segmentation Map

Control which regions become which objects.
Semantic layout control.



Depth Map

Guide the 3D structure and perspective.
ControlNet depth conditioning.



Pose Estimation

Transfer human body poses across images.
OpenPose skeleton keypoints.



Scribble / Edge

Use rough sketches or edge maps to guide composition and structure.

Classifier-Free Guidance (CFG)

Why do two people get different images from the same prompt? One reason: the guidance scale.

$$\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + w \times (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

w = guidance scale (CFG) · ϵ_{uncond} = unconditional prediction · ϵ_{cond} = text-conditioned prediction



Prompt Adherence

How closely the image follows the text prompt



Creativity & Diversity

How varied and unexpected the outputs can be



The Trade-off

Higher CFG = more faithful, less diverse

CFG is sampled at inference time — same model, different guidance = different image characteristics.

CFG Guidance Scale



LOW CFG (Creative)

HIGH CFG (Faithful)

CFG 1 – 3



Very Creative

Prompt loosely followed.
Surreal, unexpected results.
Maximum diversity.

CFG 5 – 8



Balanced

Recommended range for
most use cases. Good
balance of quality and
adherence.

CFG 10 – 15



Prompt Faithful

Strong adherence to every
word. Less creative variation
between seeds.

CFG 20+



Artifacts Risk

Oversaturated, distorted, or
'burned' outputs. Use with
caution.

Try CFG 7–8 first. Increase for more literal interpretation, decrease for more artistic freedom.

Applications of Diffusion Models



Image Generation

Text-to-image, style transfer



Image Editing

Object removal, background swap



Restoration

Inpainting, super-resolution, denoising



Video

Text-to-video, frame interpolation



Audio

Music generation, speech synthesis



3D Objects

3D asset generation, NeRF guidance



Healthcare

Medical image synthesis, augmentation



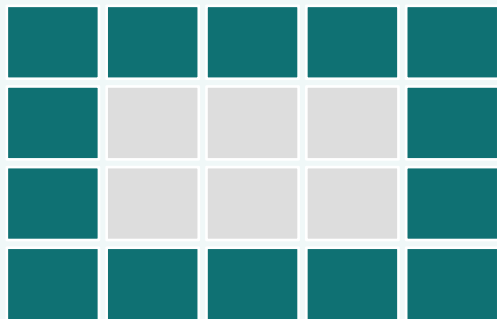
Super Resolution

Upscale low-res images to 4K+

Inpainting vs Outpainting

INPAINTING

Fill missing regions inside an image



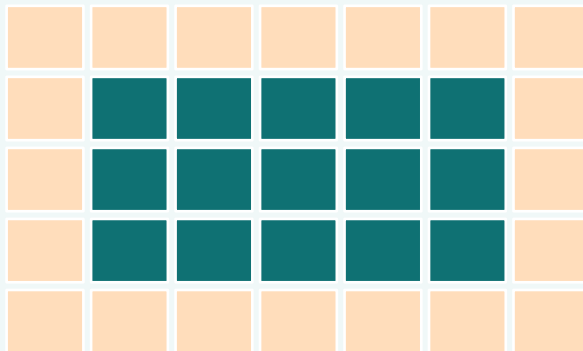
masked region

↓ Fill with diffusion

- Remove objects from scenes
- Repair damaged or masked areas
- Replace faces, clothing, backgrounds

OUTPAINTING

Extend the image beyond its borders

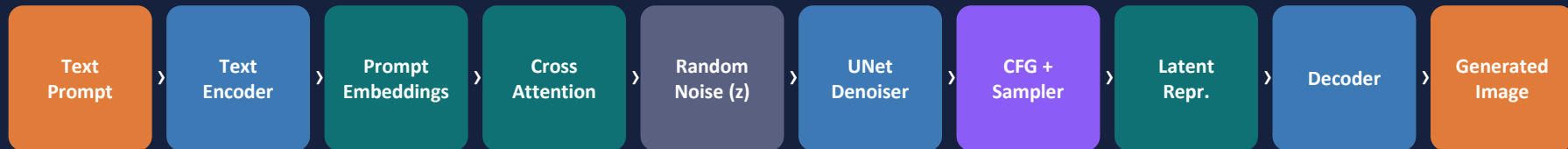


original · new extension

↓ Extend with diffusion

- Expand image canvas outward
- Generate new scene content seamlessly
- Create wide-angle or panoramic views

Complete Modern Diffusion Pipeline



Latent Diffusion

- ▶ Encode images into compact latent space
- ▶ Diffusion happens on latents (not pixels)
- ▶ 64× more efficient than pixel diffusion
- ▶ Decode latents back to high-res images

Text Conditioning

- ▶ CLIP/T5 encodes text → embeddings
- ▶ Cross-attention injects text into UNet
- ▶ Each word guides specific image regions
- ▶ Conditioning types: text, image, depth, pose

CFG & Applications

- ▶ CFG scale controls adherence vs creativity
- ▶ Recommended range: 7–8 for most tasks
- ▶ Applications: image, video, audio, 3D, medical
- ▶ Inpainting fills · Outpainting expands